

VLA-Pruner: Temporal-Aware Dual-Level Visual Token Pruning for Efficient Vision-Language-Action Inference

Ziyan Liu^{1,2} Yeqiu Chen^{1,2} Hongyi Cai¹ Tao Lin¹ Shuo Yang³ Zheng Liu⁴ Bo Zhao¹

Abstract

Vision-Language-Action (VLA) models have shown promise for embodied AI but suffer from heavy computational overhead of processing continuous visual streams. Visual token pruning—keeping salient tokens and dropping redundant ones—emerges as a mainstream technique to accelerate Vision-Language Models (VLMs), offering a natural solution. However, these VLM-specific token pruning methods rely solely on *semantic salience* metrics (e.g., prefill attention) to select tokens, while overlooking VLA models’ intrinsic *dual-system nature* of high-level semantic understanding and low-level action execution. Consequently, they bias token retention toward semantic cues, discard critical information for action generation, and significantly degrade VLA performance. To bridge this gap, we propose **VLA-Pruner**, a plug-and-play VLA token pruning method that aligns with the dual-system nature of VLA models and exploits the temporal continuity in robot manipulation. Specifically, VLA-Pruner introduces a *dual-objective importance criterion* for token retention: vision-language prefill attention for semantic relevance and action decode attention, estimated via *temporal smoothing*, for execution precision. Based on this criterion, VLA-Pruner proposes a *dual-level token selection strategy* that adaptively preserves a compact set of visual tokens for both semantic understanding and action execution under given compute budget. Experiments show that VLA-Pruner outperforms state-of-the-art approaches across multiple VLA architectures, achieving up to $1.99\times$ speedup with comparable manipulation quality. Our code is publicly available at: <https://github.com/MINT-SJTU/VLA-Pruner.git>.

¹School of AI, Shanghai Jiao Tong University ²University of Science and Technology of China ³Harbin Institute of Technology (Shenzhen) ⁴BAAI. Correspondence to: Bo Zhao <bo.zhao@sjtu.edu.cn>.

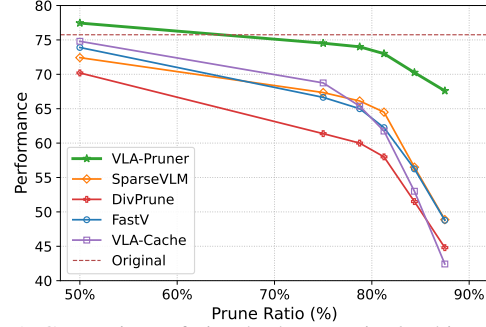


Figure 1. Comparison of visual token pruning/caching methods across various pruning/caching ratios. The plot reports the average success rate on the LIBERO (Liu et al., 2023a) benchmark. VLA-Pruner outperforms all baselines, especially at high ratios. At a 50% ratio, VLA-Pruner can improve model performance.

1. Introduction

Recent advances in robot learning have led to remarkable progress in developing generalist policies capable of acting across diverse tasks and environments (Ma et al., 2024; Firoozi et al., 2025; Zhao et al., 2025; Brohan et al., 2022; Zitkovich et al., 2023; Kim et al., 2024; Black et al., 2024; Li et al., 2024a; Chi et al., 2025). Vision-Language-Action (VLA) models (Brohan et al., 2022; Zitkovich et al., 2023; Kim et al., 2024; 2025a; Black et al., 2024; Li et al., 2024a; Lin et al., 2025a), which benefit from the multimodal understanding and transfer ability of Vision-Language Models (VLMs) (Liu et al., 2023b; Bai et al., 2023), have emerged as a promising pathway toward general-purpose robot policy. Leveraging large-scale real-world robotic datasets (O’Neill et al., 2024), VLA models can perceive visual scenes, understand natural language instructions, and directly execute low-level actions to complete complex tasks across diverse scenarios. However, in real-time deployment, VLA models must process continuous visual streams within dynamic environments and thus struggle with heavy computational overhead. To mitigate this, existing works have explored acceleration techniques such as model lightweighting (Wen et al., 2025), quantization (Park et al., 2024), and early-exit (Yue et al., 2024) for VLA. Despite effectiveness to some extent, these methods require either architectural modifications or retraining, hindered by poor generalizability.

Previous research (Chen et al., 2024; Shang et al., 2024; Alvar et al., 2025) has shown that visual representations

(i.e., visual tokens) in Vision-Language Models (VLMs) are highly redundant. Therefore, visual token pruning, which retains the most salient tokens while pruning the rest, has become a general and effective approach to improve VLM inference efficiency (Alvar et al., 2025; Chen et al., 2024; Zhang et al., 2024; Shang et al., 2024; Ye et al., 2025; Xing et al., 2024; Zou et al.) and shows promise for general VLA acceleration. These methods typically define **importance criteria** for tokens, such as attention scores (Chen et al., 2024; Zhang et al., 2024; Zou et al.), gradient information (Mao et al., 2025b;a) or feature diversity (Alvar et al., 2025) to quantify the significance of visual tokens, and less important tokens are pruned at inference time. By reducing visual redundancy, they yield substantial compute savings with minimal accuracy loss on standard VLM tasks such as image captioning (Agrawal et al., 2019; Plummer et al., 2015), VQA (Hudson & Manning, 2019; Liu et al., 2024), and video understanding (Xu et al., 2017; Li et al., 2016).

Despite recent progress, existing token pruning methods rely solely on *semantic salience* metrics measured during context prefilling—e.g., prefill attention in FastV (Chen et al., 2024) or text-to-vision cross-attention scores in SparseVLM (Zhang et al., 2024)—to rank visual tokens and prune the lowest-scoring ones. While such semantic-only criteria can be effective for VLMs, they are misaligned with the intrinsic **dual-system nature** of VLA models (Han et al., 2024; Huang et al., 2025; Sloman, 1996). Specifically, VLA inference couples high-level semantic understanding, task planning with low-level action execution, which entails distinct visual requirements between its **vision-language prefill** and **action decode** stages; correspondingly, we observe clearly **different attention patterns** across these two stages (see Sec. 3). Consequently, current methods tend to bias token retention towards semantic cues, thereby discarding critical details for action generation and significantly degrading VLA performance, especially at high pruning ratios (see Figure 1), as also observed in prior work (Xu et al., 2025). Thus, existing VLM-specific token pruning strategies are fundamentally *ill-suited* for VLA models, highlighting the urgent need for a VLA-specific method.

To bridge this gap, we propose **VLA-Pruner**, a versatile plug-and-play token pruning approach that aligns with the dual-system nature of VLA models and exploits the temporal continuity in robot manipulation. VLA-Pruner offers two principal insights for VLA token pruning. First, VLA-Pruner adopts a **dual-objective importance criterion** for visual token retention: vision-language prefill attention scores for semantic relevance (following prior works (Chen et al., 2024; Ye et al., 2025; Mao et al., 2025b; Zhang et al., 2024)) and action decode attention scores for execution precise. A critical challenge to achieve this is that decode attention is unavailable at prefill time. Fortunately, we observe that *temporal continuity* of robot manipulation (Xu et al.,

2025) manifests in VLA action attention patterns evolving: top-attended visual patches overlap substantially across consecutive timesteps (see Sec. 3). Leveraging this continuity, we practically estimate the current action-to-vision attention from recent decode attentions via statistical temporal smoothing techniques. Second, to ensure robust performance, our **dual-level token selection strategy** avoids fragile attention score fusion that incurs redundancy and bias. We draw on the *minimal-redundancy-maximal-relevance* (mRMR) principle (Peng et al., 2005) and adopt a patch-wise scheme: take the union of patches salient at either semantic or action level to maximize relevance, and then filter them by reducing redundancy, yielding a compact yet informative visual token set that meets the given compute budget. This innovative design reduces computational overhead and adaptively preserves essential information for both semantic understanding and action execution.

We summarize our contributions. (1) We reveal the dual-system property of VLA inference, observing distinct attention patterns between prefill and decode stages, which identifies the fundamental limitations of current VLM pruning methods for VLA acceleration (see Sec 3). (2) We propose VLA-Pruner, a general, training-free framework that aligns with this intrinsic duality and exploits the temporal continuity in VLA manipulation (see Sec 4). (3) Serving as a plug-and-play module for multiple VLA models, VLA-Pruner achieves state-of-the-art performance across two simulated environments, delivering up to $1.99\times$ speedup with negligible performance drops and maintaining robustness even at an 87.5% pruning ratio (see Sec 5). We also valid its real-world advantage on a 6-DoF xArm6 robot.

2. Preliminary

2.1. Related Works

Vision-Language-Action Models (VLA) Large-scale vision-language models (VLMs) have propelled multimodal learning by incorporating visual perception and language reasoning (Li et al., 2023). Building on these advances, Vision-Language-Action (VLA) models (Kim et al., 2024; Black et al., 2024; Kim et al., 2025b; Brohan et al., 2022; Zitkovich et al., 2023) introduce an action modality, enabling end-to-end motor control. These models typically adopt large VLM backbones (Touvron et al., 2023) and are fine-tuned on robotic data (O’Neill et al., 2024). For action generation, implementations range from discretizing actions into language-like tokens (e.g., OpenVLA (Kim et al., 2024)) to attaching diffusion-policy heads (e.g., π_0 (Black et al., 2024)), typically relying on *action-to-vision cross-attention* for fine-grained interaction. Despite effectiveness on robot manipulation (Liu et al., 2023a; Li et al., 2024b), VLA models remain computationally intensive, hindering real-time deployment on resource-constrained platforms.

Visual Token Pruning for VLMs Visual token pruning (Chen et al., 2024; Zhang et al., 2024; Alvar et al., 2025; Shang et al., 2024; Ye et al., 2025) is a widely adopted strategy to reduce visual redundancy in VLMs. One line of these methods defines *semantic importance criteria* to rank and prune visual tokens, e.g., FastV (Chen et al., 2024) (using prefill attention), SparseVLM (Zhang et al., 2024) (using text-to-vision attention), DivPrune (Alvar et al., 2025) (using diversity-driven selection), and HoloV (Zou et al.) (using crop-wise attention and diversity). Another line of *calibration-based* approaches (e.g., FitPrune (Ye et al., 2025), VTW (Lin et al., 2025b)) select pruning layers/ratios by analyzing model outputs on a calibration set. However, the first line relies mainly on semantic salience and overlooks the action-dependent requirements of VLA (see Sec. 3). The second line depends on offline calibration that is unfriendly to resource-constrained VLA deployment and still hinges on semantic salience. These limitations motivate a VLA-specific token pruning method.

Training-Free Acceleration for VLA Models Visual token pruning has recently been integrated into training-free VLA acceleration frameworks (Yu et al., 2025). To name a few, EfficientVLA (Yu et al., 2025), SP-VLA (Li et al., 2025), and SpecPrune-VLA (Wang et al., 2025) perform structured acceleration that incorporates token pruning. However, these methods still use semantic salience to retain tokens. Thus, they rely on additional modules (e.g., diffusion-feature cache (Yang et al., 2025), lightweight generation module (Li et al., 2025), or layer-reduction (Yang et al., 2025; Wang et al., 2025)) to balance performance and speed, limiting generalizability and effectiveness. A recent method, VLA-Cache (Xu et al., 2025), exploits temporal continuity by caching static visual token features, opening a promising direction. However, its cache-based mechanism is less efficient than direct pruning and still depends on text-to-vision cross-attention to preserve task-relevant tokens. The above limitations stem from overlooking the dual-system nature of VLA models (see Sec. 3). In this paper, we show that visual token pruning can achieve significant VLA inference acceleration without sacrificing performance.

2.2. Background of VLA Inference

VLA Inference. Vision-Language-Action (VLA) models process sequential inputs of the form $\{V^t, L^t\}$ into A^t across timesteps t . V^t represents the input visual observations, L^t denotes language task instructions, and $A^t = \{a_1^t, \dots, a_N^t\}$ denotes the predicted action tokens, which are detokenized into executable robotic actions. The text input is tokenized into N text tokens $\mathbf{E}_\tau^t = \{\tau_1^t, \dots, \tau_N^t\}$. The vision encoder processes V^t into image features, which are projected into M visual tokens $\mathbf{E}_v^t = \{v_1, \dots, v_M\}$. These tokens, along with a proprioceptive token representing robot state, form the Transformer input. Since self-attention scales quadratically with token counts, the M (generally $M \gg N$)

visual tokens dominate inference cost.

VLA Attention Breakdown. Like VLMs, VLA models rely on the attention mechanism of transformer for token interactions. Without loss of generality, we describe the single-head attention below. At each layer l , the model takes previous-layer hidden states of visual tokens $\mathbf{H}_v^{l-1} \in \mathbb{R}^{M \times d}$ and textual tokens $\mathbf{H}_\tau^{l-1} \in \mathbb{R}^{N \times d}$ as inputs. *For mathematical simplicity, we omit time/layer superscripts and treat the proprioceptive token as part of the vision-language context.* These inputs are transformed into queries $\mathbf{Q}$, keys $\mathbf{K}$, and values $\mathbf{V}$ using linear projections. Then, the shared vision-language context for attention and interaction is formed as: $\mathbf{K}_{vl} \in \mathbb{R}^{(N+M) \times d_k}$, $\mathbf{V}_{vl} \in \mathbb{R}^{(N+M) \times d_v}$. We break down VLA inference into two stages:

(1) Vision-Language prefill Stage. In this stage, queries come from the text and vision tokens: $\mathbf{Q}_{vl} \in \mathbb{R}^{(N+M) \times d_k}$. The attention matrix is computed by

$$\mathcal{A}_{vl} = \text{Softmax}\left(\frac{\mathbf{Q}_{vl}\mathbf{K}_{vl}^\top}{\sqrt{d_k}}\right). \quad (1)$$

Attention score for each input visual patch is computed as

$$\mathcal{S}_{vl}[m] = \frac{1}{M+N} \sum_{i=1}^{M+N} \mathcal{A}_{vl}[i, m], \quad m = 1, \dots, M. \quad (2)$$

(2) Action decode Stage. In this stage, queries are action tokens $\mathbf{Q}_{act} \in \mathbb{R}^{K \times d_k}$ ($K = 1$ for autoregressive (Kim et al., 2024), $K = \hat{N}$ for chunk-based (Black et al., 2024)). Attention is computed as

$$\mathcal{A}_{act} = \text{Softmax}\left(\frac{\mathbf{Q}_{act}\mathbf{K}_{vl}^\top}{\sqrt{d_k}}\right). \quad (3)$$

For the autoregressive case, attention vectors from $\hat{N}$ decoding steps are concatenated into the final attention matrix. The action-to-vision attention score in this stage is

$$\mathcal{S}_{act}[m] = \frac{1}{\hat{N}} \sum_{i=1}^{\hat{N}} \mathcal{A}_{act}[i, m], \quad m = 1, \dots, M. \quad (4)$$

3. Motivation

We systematically delve into the intrinsic characteristics of VLA inference (Figure 2). Our findings identify the fundamental limitations of current VLM-specific token pruning methods for VLA models, which motivate our VLA-Pruner.

Motivation 1: Visual token prefilling dominates computational overhead for VLA models. As noted in previous works (Zhang et al., 2024; Chen et al., 2024), incorporation of visual tokens inevitably introduces substantial memory and computational overhead. For VLA models, the number of processed visual tokens (typically $256 \times n$ per frame, where n is the number of camera views) is often an order of magnitude larger than that of text tokens (~ 30 – 50) and action tokens (~ 7 – 56), making input visual stream prefilling the primary contributor to the computational cost. Since

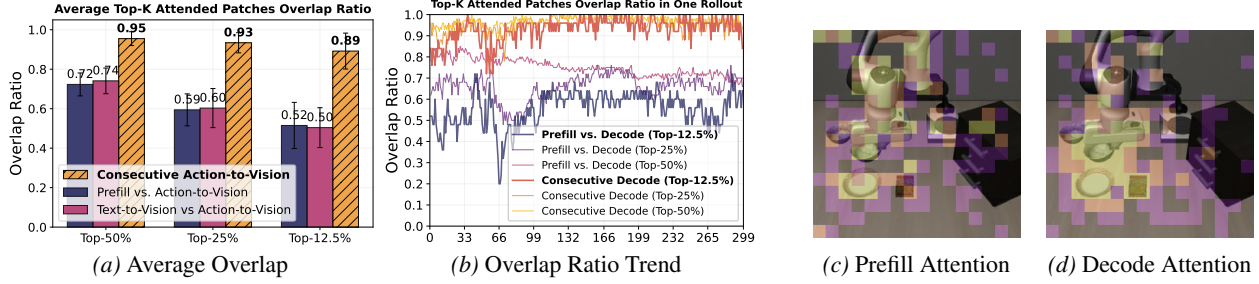


Figure 2. **Distinct attention patterns across VLA inference.** (a–b) Overlap ratios of Top- k attended patches: (i) between vision–language prefill and action decode (Top- $k(S_{vl})$ vs. Top- $k(S_{act})$), and (ii) between consecutive action-decode timesteps (Top- $k(S_{act}^t)$ vs. Top- $k(S_{act}^{t-1})$). We show average values (a) and a representative rollout (b). (c–d) Visualization of prefill (c) and action decode (d) attention over the same frame; top 12.5% (yellow), 25% (orange), and 50% (purple) patches are overlaid. Prefilling shows broad semantic coverage, while action decoding is locally focused. The results reveal VLA model’s **dual-system nature** that token pruning must handle.

visual representations are highly redundant (Alvar et al., 2025; Chen et al., 2024; Zhang et al., 2024), visual token pruning shows great promise for VLA acceleration.

Motivation 2: Visual token pruning must account for the inherent dual-system nature of VLA models. The efficacy of VLA models stems from their inherent dual-system ability to integrate high-level semantic understanding and task planning with low-level action execution (Firoozi et al., 2025; Han et al., 2024; Huang et al., 2025; Sloman, 1996; Cui et al., 2025). Our key insight is that this inherent duality manifests directly in VLA model inference, resulting in distinct visual requirements for its two stages: vision-language prefill and action decode. To investigate this, we analyzed the attention distributions of OpenVLA (Kim et al., 2024) on the LIBERO benchmark (Liu et al., 2023a) (Figure 2). The results reveal that VLA inference demonstrates **distinct attention patterns**: prefill attention exhibits a broad, semantic distribution, while action decode attention shows localized focus essential for precise motor control, as exemplified in Figure 2c and Figure 2d. Quantitatively, the overlap ratio of top-attended visual tokens between vision-language prefilling and action decoding averages $\sim 50\%$ and often falls below 30% (Figure 2). Attention scores are averaged across all Transformer layers. Existing pruning strategies overlook this crucial attention divergence, thereby risking the premature removal of tokens vital for action generation but less salient during prefilling, particularly at high compression ratios. This oversight is critical, since removing even a few high-attention tokens is known to severely degrade performance (Zhang et al., 2023; Ge et al., 2023). We provide a qualitative illustration in Figure 7.

Motivation 3: VLA token pruning should exploit temporal continuity in robotic manipulation. Since vision token prefilling dominates computational overhead (Motivation 1), token pruning must operate during early-layer prefilling (Chen et al., 2024). However, while an ideal strategy should consider both semantic and action-level token importance (Motivation 2), action decode attention scores (practical proxy for action-level importance) are unavail-

able at prefill time. Fortunately, VLA robot manipulation exhibits *temporal continuity* (Xu et al., 2025). We observe that action-to-vision attention maps show strong short-term temporal correlation: top-attended visual tokens during action decoding at timesteps t and $t-1$ show consistently high overlap ratios. Figure 2 shows averaged ratios and a representative rollout. This continuity makes statistical temporal smoothing techniques (Winters, 1960; Gardner, 1985; Hyndman & Athanasopoulos, 2018) a practical and effective proxy for estimating current action-to-vision attention.

4. Methodology

4.1. Problem Formulation

We define the VLA token pruning problem as follows: given a set of visual tokens $\mathbf{E}_v$ with $|\mathbf{E}_v| = M$ and a target subset size $\tilde{M} < M$, the goal is to select a subset $\tilde{\mathbf{E}}_v$ that preserves information critical for VLA inference. We define a mapping function f selects an index subset $\tilde{\mathcal{I}} \subseteq \{1, \dots, M\}$ with $|\tilde{\mathcal{I}}| = \tilde{M}$, and constructs the retained visual tokens as $\tilde{\mathbf{E}}_v = f(\mathbf{E}_v) = \{\mathbf{E}_v[i] : i \in \tilde{\mathcal{I}}\}$. The objective is to identify a f that minimizes the difference in the model’s output before and after pruning:

$$\min_f \mathcal{L}(\mathcal{P}, \tilde{\mathcal{P}}) \quad \text{s.t.} \quad |f(\mathbf{E}_v)| = \tilde{M} \quad (5)$$

where $\mathcal{P} = P(A|\mathbf{E}_\tau, \mathbf{E}_v)$ and $\tilde{\mathcal{P}} = P(A|\mathbf{E}_\tau, f(\mathbf{E}_v))$. $P(\cdot)$ denotes the conditional generation probability of VLA model. $\mathcal{L}$ measures the difference in the model’s output with and without pruning, and $\tilde{M}$ indicates the number of retained tokens. To formalize the intuition behind **Motivation 2**, we decompose $\mathcal{P}$ into vision-language prefill P_{vl} and action decode P_{act} based on hidden states $\mathbf{Z}_\tau, \mathbf{Z}_v$:

$$\begin{aligned} \mathcal{P} &= P_{vl}(\mathbf{Z}_\tau, \mathbf{Z}_v | \mathbf{E}_\tau, \mathbf{E}_v) \cdot P_{act}(A | \mathbf{Z}_\tau, \mathbf{Z}_v), \\ \tilde{\mathcal{P}} &= \tilde{P}_{vl}(\tilde{\mathbf{Z}}_\tau, f(\tilde{\mathbf{Z}}_v) | \mathbf{E}_\tau, f(\mathbf{E}_v)) \cdot \tilde{P}_{act}(A | \tilde{\mathbf{Z}}_\tau, f(\tilde{\mathbf{Z}}_v)). \end{aligned}$$

Here, f sparsifies both the input $\mathbf{E}_v$ and corresponding hidden states $\mathbf{Z}_v$. For VLA inference, relying on single-level objectives—either semantic-level $\mathcal{L}(P_{vl}, \tilde{P}_{vl})$ or action-level $\mathcal{L}(P_{act}, \tilde{P}_{act})$ —fails to preserve the joint probability $\mathcal{P}$. To address this, we introduce VLA-Pruner.

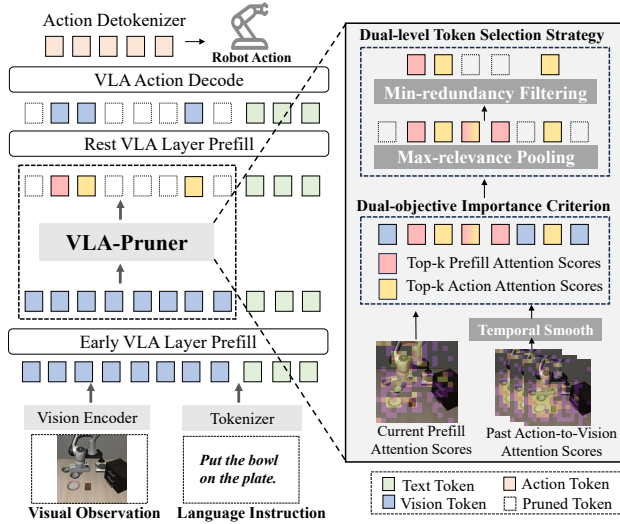


Figure 3. **Overview of VLA-Pruner** (illustrated with budget $k=3$). It unites tokens salient at semantic or action level, then filters redundancy to obtain a compact subset that preserves essential information for both semantic understanding and action execution.

4.2. The Proposed VLA-Pruner

VLA-Pruner adopts a *dual-objective importance criterion* and a corresponding *dual-level token selection strategy* for VLA token pruning. The overall pipeline is illustrated in Figure 3 and detailed in Algorithm 1.

4.2.1. DUAL-OBJECTIVE IMPORTANCE CRITERION

For VLMs, previous works (Chen et al., 2024; Zhang et al., 2024) have demonstrated that prefill or text-to-vision attention scores effectively quantify the importance of vision tokens. For VLA models, as discussed before (Sec. 3), both semantic-level and action-level visual requirements should be satisfied. Thus, VLA-Pruner adopts a *dual-objective importance criterion*: (i) prefill attention scores $\mathcal{S}_{\text{vl}}^t$ (Eq. 2) to quantify semantic-level relevance and (ii) action attention scores $\mathcal{S}_{\text{act}}^t$ (Eq. 3) to quantify action-level importance.

Attention Estimation via Temporal Smoothing. As discussed in Motivation 3, we build on the *temporal continuity* of VLA manipulation to estimate current action attention scores $\mathcal{S}_{\text{act}}^t$ over image patches. This formulates a *time-series forecasting task* (Box et al., 2015), where $\mathcal{S}_{\text{act}}^t$ is estimated from historical trend $\{\mathcal{S}_{\text{act}}^{t-1}, \mathcal{S}_{\text{act}}^{t-2}, \dots\}$. We adopt the Exponential Moving Average (EMA) technique (Hyndman & Athanasopoulos, 2018; Gardner Jr, 2006; Gardner, 1985) to ensure stable estimation and adaptability to dynamic shifts. EMA reduces noise, captures temporal trends by emphasizing recent observations while discounting older ones, and has been widely applied in domains such as economics, meteorology, and robotic manipulation (Contreras et al., 2003; Mhammed et al., 2016). Directly applying EMA to our task takes this form:

$$\hat{\mathcal{S}}_{\text{act}}^t = (1 - \alpha)\hat{\mathcal{S}}_{\text{act}}^{t-1} + \alpha\mathcal{S}_{\text{act}}^{t-1} \quad (6)$$

where $\alpha \in [0, 1]$ is the smoothing factor, $\hat{\mathcal{S}}_{\text{act}}^{t-1}$ denotes the

smoothed prediction at the previous timestep, and $\mathcal{S}_{\text{act}}^{t-1}$ represents the corresponding observation result. For VLA manipulation, where short-term context is more critical (Xu et al., 2025), we adapt the standard EMA into a **Decaying Window Average mechanism**. Formally, we smooth action attention scores from previous w steps and compute the estimated score for current step $\hat{\mathcal{S}}_{\text{act}}^t$ as their weighted average:

$$\hat{\mathcal{S}}_{\text{act}}^t = \frac{\sum_{i=1}^w \gamma^i \mathcal{S}_{\text{act}}^{t-i}}{\sum_{i=1}^w \gamma^i} \quad (7)$$

where w defines the window size, and $\gamma \in [0, 1]$ controls the decay rate. This formulation provides a simple yet effective mechanism to estimate current action attention scores. For brevity, we omit time superscript t in following discussion.

Remark 1. *Temporal smoothing can reliably estimate action attention dynamic trends but may fail under abrupt changes, such as target switching. For instance, in the LIBERO-Long task “put both the alphabet soup and the tomato sauce in the basket”, attention from putting the soup does not ground the tomato sauce well. Potential remedies include lightweight predictors or mechanisms to bypass pruning when large discrepancies occur. To ensure generalizability and flexibility, we instead tackle this fundamental challenge through our dual-level token selection strategy.*

4.2.2. DUAL-LEVEL TOKEN SELECTION STRATEGY

Based on the dual-objective token importance criterion, an intuitive approach to retain tokens is: normalizing semantic and action attention scores ($\mathcal{S}_{\text{vl}}$ and $\hat{\mathcal{S}}_{\text{act}}$), taking a weighted sum, and selecting the top- $\bar{M}$ tokens by the weighted score. However, this scheme suffers from critical limitations. **First**, it introduces a sensitive weighting hyperparameter that must be carefully tuned, compromising robustness and generalizability. **Second**, it ignores redundancy both within and across stages. On one hand, as noted in prior work (Alvar et al., 2025), attention-based pruning often retains similar tokens. On the other hand, score-level fusion tends to retain tokens that appear salient in both prefilling and decoding, yet not essential to either stage. For example, in a robotic arm, intermediate arm segments may appear salient across both stages, whereas the end-effector (critical for execution) can be undervalued. **Third**, it implicitly assumes that the action attention estimate $\hat{\mathcal{S}}_{\text{act}}$ is always reliable, which may not hold under abrupt attention shifts (see Remark 1).

To avoid these limitations, we draw on the *minimal-Redundancy-Maximal-Relevance* (mRMR) principle (Peng et al., 2005), an information-theoretic paradigm for feature selection. mRMR proposes to combine both Max-Relevance and Min-Redundancy principles, keeping the relevant features for the task while reducing the redundant features. Inspired by this principle, our proposed dual-level token selection strategy follows a patch-wise **Combine-then-Filter** paradigm. We organize the procedure into three phases:

(1) Dual-level Top-k Selection. We begin by constructing two candidate token subsets via dual-level top-k selection:

$$\mathcal{C}_{\text{vl}} = \text{Top-}\tilde{M}(\{\mathcal{S}_{\text{vl}}[i]\}_{i=1}^{\tilde{M}}), \mathcal{C}_{\text{act}} = \text{Top-}\tilde{M}(\{\hat{\mathcal{S}}_{\text{act}}[i]\}_{i=1}^{\tilde{M}}),$$

where $\tilde{M}$ is the target token budget. This phase identifies salient tokens (indices) for high-level semantic understanding ($\mathcal{C}_{\text{vl}}$) and low-level action execution ($\mathcal{C}_{\text{act}}$), respectively.

(2) Relevance-Maximization Pooling. We then take the union of these two subsets to form a candidate pool:

$$\mathcal{C}_{\text{dual}} = \mathcal{C}_{\text{vl}} \cup \mathcal{C}_{\text{act}}.$$

This subset $\mathcal{C}_{\text{dual}}$ constructs a candidate pool that contains tokens essential for dual-level information requirements, maximizing two-stage task relevance. Due to attention divergence (Motivation 2), $|\mathcal{C}_{\text{dual}}|$ is generally larger than $\tilde{M}$.

(3) Redundancy-Minimization Filtering. This phase aims to reduce feature redundancy within the candidates $\mathcal{C}_{\text{dual}}$ to satisfy the target budget $\tilde{M}$. We achieve this by maximizing the diversity of the final selected subset. Following prior work (Alvar et al., 2025), we formulate it as a Max-Min Diversity Problem (MMDP) (Porumbel et al., 2011) within candidate pool $\mathcal{C}_{\text{dual}}$, where the goal is to find a subset $\tilde{\mathcal{C}} \subset \mathcal{C}_{\text{dual}}$ of size $\tilde{M}$ that maximizes the minimum pairwise distance between its elements:

$$\tilde{\mathcal{C}} = \underset{\mathcal{C}}{\operatorname{argmax}} \left[\min_{i,j \in \mathcal{C}} d(v_i, v_j) : \forall \mathcal{C} \subset \mathcal{C}_{\text{dual}}, |\mathcal{C}| = \tilde{M} \right], \quad (8)$$

where $\mathcal{C}$ is any subset of $\mathcal{C}_{\text{dual}}$ with $\tilde{M}$ elements, and $v_i, v_j \in \mathbf{E}_v$ are input image patch embeddings indexed by any $i, j \in \mathcal{C}$. The distance $d(\cdot, \cdot)$ is defined as the cosine distance:

$$d(v_i, v_j) = 1 - \frac{v_i \cdot v_j}{\|v_i\| \|v_j\|}. \quad (9)$$

A solution for the MMDP problem 8 is a subset of $\mathcal{C}_{\text{dual}}$ that minimizes redundancy between elements. MMDP can be solved by several exact and heuristic methods (Resende et al., 2010; Martí et al., 2022). To facilitate efficiency, we use a greedy algorithm that iteratively add tokens whose minimum distance to the retained set is maximal, until reaching $\tilde{M}$ elements. To escape local optima, we initialize the first token with maximal second-nearest distance. We detail the step (3) in Algorithm 2 (see Appendix A).

Remark 2. Our method is a heuristic solution for mRMR objective: the first two steps maximize task relevance, while the final step minimizes feature redundancy. The challenge in Remark 1 is addressed by grounding abrupt shifts by pre-fill attention and retaining them via diversity maximization.

Remark 3. Step (3) shares similarities with DivPrune (Alvar et al., 2025), which reduces feature redundancy through diversity-based pruning. However, DivPrune does not incorporate the Max-relevance step, and thus fails to identify pivotal visual tokens. By combining both Max-relevance and Min-redundancy principles, we retain token subsets that are both diverse and better matched to dual-level visual information requirements for VLA inference.

4.3. Implementation Details.

VLA-Pruner prunes visual tokens at Transformer layer K (K set to 3 (Chen et al., 2024)). It selects tokens using last-layer prefill attention and temporal smoothing of action-to-vision attention averaged over the latter half layers. Pruned tokens are dropped from the rest prefill layers. For decaying window average, window size w and decay rate γ are empirically set as 3 and 0.8. We warm-start VLA-Pruner with w steps to record sufficient attention history. VLA-Pruner is training-free, serving as a plug-and-play acceleration module for VLA architectures with action-to-vision cross-attention, a mechanism used by state-of-the-art VLA models (Kim et al., 2024; 2025b; Black et al., 2024), including (i) autoregressive policies (Kim et al., 2024; Brohan et al., 2022; Zitkovich et al., 2023) (e.g., OpenVLA (Kim et al., 2024)), (ii) action-chunk decoders (OpenVLA-OFT (Kim et al., 2025b)), and (iii) diffusion-head policies such as π_0 (Black et al., 2024), for which we average action attention over flow-matching steps. We provide theoretical analysis of computational complexity in Appendix A.2.

5. Experiments

We evaluate VLA-Pruner in both simulation and real-world settings. We evaluate VLA-Pruner on open-source VLA models: OpenVLA (Kim et al., 2024), OpenVLA-OFT (Kim et al., 2025b) and π_0 (Black et al., 2024). All experiments are conducted on an NVIDIA RTX 4090 GPU.

5.1. Experimental Setup

(1) Baselines. We compare VLA-Pruner with a comprehensive suite of training-free accelerators under given compute budgets. Baselines include state-of-the-art token pruning methods (FastV (Chen et al., 2024), SparseVLM (Zhang et al., 2024), and DivPrune (Alvar et al., 2025)) and a VLA-specific method VLA-Cache (Xu et al., 2025). We provide more details of baselines and VLA models in Appendix B.1. EfficientVLA (Yang et al., 2025) uses token pruning strategies similar to prior work (using attention and diversity). We include results of it and HoloV (Zou et al.) in Appendix B.3.

(2) Evaluation Protocol. All methods are evaluated under identical given pruning budgets. To ensure a comprehensive comparison, our experiments include diverse pruning ratios (50%, 75%, 87.5%). For VLA-Cache, we set the same token reuse ratio. Our evaluation metrics mainly include: task success rate (%), inference latency (ms) and FLOPs(T).

(3) Evaluation Benchmarks. We introduce the evaluation benchmarks, with more details presented in Appendix B.2: **LIBERO**. The LIBERO benchmark (Liu et al., 2023a) comprises four suites (*Spatial*, *Object*, *Goal*, and *Long*) that probe complementary aspects of manipulation generalization. Each suite contains 10 distinct tasks with 50 evaluation episodes per task (500 total episodes per suite). Following OpenVLA (Kim et al., 2024) and OpenVLA-OFT (Kim et al., 2025b), we adopt their standard evaluation setup (ac-

Table 1. Performance of VLA-Pruner on OpenVLA and OpenVLA-OFT across LIBERO at 50%, 25%, and 12.5% vision-token retention. For each method, the first line shows average success rates(%) on *Spatial/Object/Goal/Long*; the second line shows relative accuracy vs. vanilla (100%) or speedup ratio ($\times$). We also report FLOPs (T), inference latency (ms/action; ms/action-chunk).

Method	OpenVLA							OpenVLA-OFT						
	Spatial	Object	Goal	Long	Acc.(%) $\uparrow$	FLOPs(T) $\downarrow$	Latency(ms) $\downarrow$	Spatial	Object	Goal	Long	Acc.(%) $\uparrow$	FLOPs(T) $\downarrow$	Latency(ms) $\downarrow$
<i>Upper Bound (100%)</i>														
Vanilla	87.6	84.6	78.6	52.2	100.0	1.906	236.41	95.8	98.7	96.3	90.7	100.0	3.903	135.78
	100%	100%	100%	100%	(—)	100.00%	1.00 $\times$	100%	100%	100%	100%	(—)	100%	1.00 $\times$
<i>Retain 50% Tokens ($\downarrow$ 50%)</i>														
FastV	86.2	81.6	77.2	50.6	97.43	1.136	172.32	94.6	96.8	92.7	87.0	97.26	2.219	88.23
	98.4%	96.5%	98.2%	96.9%	($\downarrow$ 2.57)	59.60%	1.37 $\times$	98.7%	98.1%	96.3%	95.9%	($\downarrow$ 2.74)	56.85%	1.54 $\times$
SparseVLM	85.6	80.5	75.0	48.6	95.44	1.155	175.77	94.1	93.7	91.2	85.1	95.37	2.289	90.22
	97.7%	95.2%	95.4%	93.1%	($\downarrow$ 4.56)	60.60%	1.34 $\times$	98.3%	94.9%	94.7%	93.8%	($\downarrow$ 4.63)	58.65%	1.50 $\times$
DivPrune	82.6	78.8	71.8	47.6	92.64	1.105	173.88	90.8	91.1	89.9	83.1	92.61	2.126	88.01
	94.3%	93.1%	91.3%	91.2%	($\downarrow$ 7.36)	57.97%	1.36 $\times$	94.8%	92.3%	93.3%	91.7%	($\downarrow$ 7.39)	54.47%	1.54 $\times$
VLA-Cache	87.1	80.7	78.6	51.8	98.48	1.384	192.20	95.4	96.0	96.7	90.2	99.09	2.730	101.01
	99.4%	95.4%	100.0%	99.2%	($\downarrow$ 1.52)	72.61%	1.23 $\times$	99.6%	97.3%	100%	99.4%	($\downarrow$ 0.91)	69.95%	1.34 $\times$
VLA-Pruner	88.2	85.8	79.4	56.4	102.45	1.139	178.38	97.3	98.6	96.8	92.6	101.05	2.234	92.86
	100.7%	101.4%	101.0%	108.0%	($\uparrow$ 2.45)	59.76%	1.33 $\times$	101.6%	99.9%	100.5%	102.1%	($\uparrow$ 1.05)	57.24%	1.46 $\times$
<i>Retain 25% Tokens ($\downarrow$ 75%)</i>														
FastV	81.6	69.6	71.6	43.8	87.62	0.757	141.25	87.8	81.8	87.6	74.6	87.48	1.401	72.73
	93.2%	82.3%	91.1%	83.9%	($\downarrow$ 12.38)	39.72%	1.67 $\times$	91.7%	82.9%	91.0%	82.3%	($\downarrow$ 12.52)	35.89%	1.87 $\times$
SparseVLM	83.4	72.8	67.6	45.6	88.67	0.772	144.12	89.8	84.8	82.7	79.1	88.63	1.459	74.71
	95.2%	86.1%	86.0%	87.4%	($\downarrow$ 11.33)	40.50%	1.64 $\times$	93.7%	85.9%	85.9%	87.2%	($\downarrow$ 11.37)	37.38%	1.82 $\times$
DivPrune	77.4	61.3	65.4	41.4	80.96	0.743	142.12	83.9	72.6	79.3	72.5	80.76	1.389	72.30
	88.4%	72.5%	83.2%	79.3%	($\downarrow$ 19.04)	38.98%	1.66 $\times$	87.6%	73.6%	82.3%	79.9%	($\downarrow$ 19.24)	35.58%	1.88 $\times$
VLA-Cache	78.1	73.2	70.2	45.5	88.08	0.961	164.17	85.6	85.3	84.1	80.3	88.11	1.938	85.02
	89.2%	86.5%	89.3%	87.2%	($\downarrow$ 11.92)	50.42%	1.44 $\times$	89.4%	86.4%	87.3%	88.5%	($\downarrow$ 11.89)	49.65%	1.60 $\times$
VLA-Pruner	85.4	82.5	78.4	51.8	98.48	0.758	144.81	93.5	96.2	95.2	90.2	98.37	1.420	75.64
	97.5%	97.5%	99.7%	99.2%	($\downarrow$ 1.52)	39.77%	1.63 $\times$	97.6%	97.5%	98.9%	99.5%	($\downarrow$ 1.63)	36.38%	1.80 $\times$
<i>Retain 12.5% Tokens ($\downarrow$ 87.5%)</i>														
FastV	62.0	58.5	55.8	18.8	63.08	0.568	125.76	60.6	59.2	60.6	28.6	61.64	1.082	66.11
	70.8%	69.1%	71.0%	36.0%	($\downarrow$ 36.92)	29.80%	1.88 $\times$	63.3%	60.0%	62.9%	31.5%	($\downarrow$ 38.36)	27.72%	2.05 $\times$
SparseVLM	65.8	55.3	55.4	19.0	63.20	0.581	128.77	73.1	65.0	67.5	32.8	62.11	1.136	67.65
	75.1%	65.4%	70.5%	36.4%	($\downarrow$ 36.80)	30.48%	1.84 $\times$	76.3%	65.8%	70.1%	36.1%	($\downarrow$ 37.89)	29.11%	2.01 $\times$
DivPrune	55.4	54.8	51.4	17.6	58.06	0.569	125.36	61.4	61.8	64.9	30.8	56.83	1.059	66.46
	63.2%	64.8%	65.4%	33.7%	($\downarrow$ 41.94)	29.28%	1.89 $\times$	64.1%	62.6%	67.4%	34.0%	($\downarrow$ 43.17)	27.13%	2.04 $\times$
VLA-Cache	52.5	50.1	52.0	15.1	54.79	0.710	145.93	57.3	58.7	64.8	26.2	53.88	1.373	76.15
	59.9%	59.2%	66.2%	28.9%	($\downarrow$ 45.21)	37.25%	1.62 $\times$	59.8%	59.5%	67.3%	28.9%	($\downarrow$ 46.12)	35.18%	1.78 $\times$
VLA-Pruner	80.2	78.4	69.0	42.8	88.90	0.571	129.01	88.1	87.6	84.9	68.8	88.27	1.096	68.95
	91.6%	92.7%	87.8%	82.0%	($\downarrow$ 11.10)	29.96%	1.83 $\times$	91.9%	91.7%	88.1%	81.3%	($\downarrow$ 11.73)	28.08%	1.99 $\times$

tion chunks as 8 for OpenVLA-OFT) and official weights to ensure comparability. To verify cross-architecture generality, we also include results of diffusion-head based model π_0 (Black et al., 2024). (see Appendix B.4). **SIMPLER.** The SIMPLER simulators (Li et al., 2024b) provide two complementary settings aimed at closing sim-to-real gaps: visual matching (VM) and variant aggregation (VA). We report evaluation results on three tasks and their corresponding settings: *Move Near*, *Pick Coke Can* (under VA), and *Open/Close Drawer* (under VM). We choose these task–setting pairs where the vanilla OpenVLA attains non-trivial success rates, ensuring a reasonable comparison. **Real Robot Evaluation.** To assess VLA-Pruner’s performance in real-world scenarios, we conduct experiments using a 6-DoF xArm6 robotic arm equipped with a parallel gripper. We run four manipulation tasks involving diverse objects (100 trials for each task): (1) *Can Stack*: grasp the can and stack it on top of the box; (2) *Cup Pour*: pick up the cup and pour the small balls inside the cup into the box; (3) *Cube Place*: grasp the cube and place it into the box; (4) *Bottle Place*: grasp the bottle and place it into the box.

5.2. Main Results

Results on LIBERO. We evaluate VLA-Pruner on OpenVLA (fine-tuned) and OpenVLA-OFT for LIBERO benchmark. Table 1 reports *success rate* on the four suites under token-retention ratios of 50%, 25%, and 12.5%. We also

Method	Move Near	Pick Coke Can	Open/Close Drawer	Overall
OpenVLA	54.0% (100%)	52.8% (100%)	49.5% (100%)	52.1% (100%)
FastV	38.7% (71.7%)	41.9% (79.4%)	33.7% (68.1%)	38.1% (73.1%)
VLA-Cache	41.2% (76.3%)	40.6% (76.9%)	38.8% (78.4%)	40.2% (77.2%)
VLA-Pruner	52.4% (97.0%)	50.1% (94.9%)	48.8% (98.6%)	50.4% (96.8%)

Table 2. Success rates of VLA-Pruner on SIMPLER tasks and overall performance at 75% token pruning ratio. We present performance preservation ratio relative to original OpenVLA.

report a *relative accuracy* metric that measures pruning-induced error, defined as the performance preservation ratio (vanilla model as 100%). VLA-Pruner achieves the best success rate and relative accuracy across all settings among training-free accelerators. Even at a 12.5% retention ratio, it maintains 88.9% and 88.27% relative accuracy on OpenVLA and OpenVLA-OFT, respectively, outperforming baselines by up to 34.39%. At a 50% retention ratio, VLA-Pruner can even improve success rate, thanks to precise noise filtering that stabilizes policy execution. This is evident on *LIBERO-Long*, where it achieves 4.2% success rate improvement. DivPrune benefits VLM inference but degrades VLA performance since robot manipulation needs more localized visual details. VLA-Cache achieves competitive results due to temporal awareness but fails at high compression ratios because its text-to-vision attention selection strategy is coarse-grained. Figure 4 visualizes more details of VLA-Pruner performance varying under different pruning ratios. We present results on π_0 in Appendix B.4.

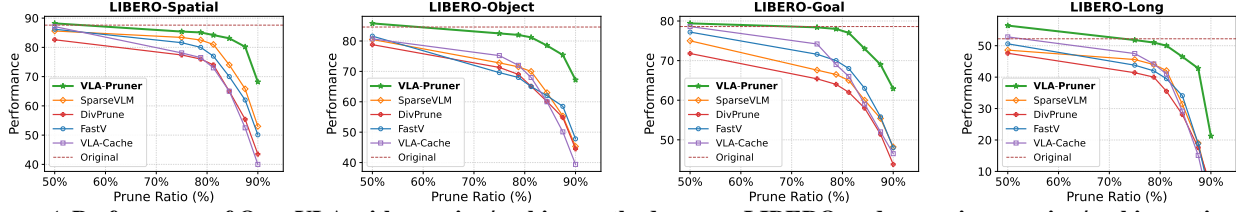


Figure 4. Performance of OpenVLA with pruning/caching methods across LIBERO under varying pruning/caching ratio. The horizontal axis represents the pruning/caching ratios of visual tokens, and the vertical axis shows the success rates. VLA-Pruner performs best, especially as ratio increases.

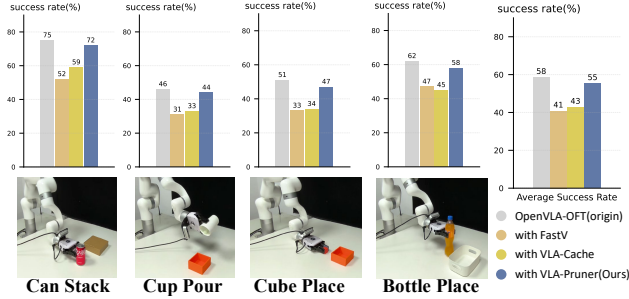


Figure 5. Performance of VLA-Pruner on OpenVLA-OFT for real-robot tasks under a 75% prune ratio. We use a 6-DoF xArm6 robotic arm. VLA-Pruner achieves superiority in preserving model performance, demonstrating its practical advantage.

Results on SIMPLER. We evaluate cross-environment generalization on the SIMPLER environment. Table 2 compares VLA-Pruner with acceleration baselines on OpenVLA at 75% compression ratio. VLA-Pruner best preserves accuracy, showing robust transfer across environments.

Results on Real Robot. We evaluate real-world performance on a physical 6-DoF xArm6 arm controlled by OpenVLA-OFT. Figure 5 reports results under 75% token-pruning ratio. VLA-Pruner attains the highest relative accuracy, underscoring its practicality for on-robot deployment.

Efficiency Analysis. We report inference efficiency metrics in Table 1, including inference latency (ms/action or ms/action-chunk) and FLOPs(T). At same pruning ratios, VLA-Pruner achieves comparable speedups to visual token-pruning methods. Its latency is ~ 5 ms higher than FastV (due to distance computations in redundancy filtering), yet it achieves significantly better performance. At the 25% retention ratio, VLA-Pruner outperforms token pruning baselines at 50% retention, while being $\sim 16\%$ faster and using $\sim 20\%$ fewer FLOPs. VLA-Pruner is faster and more compute-efficient than VLA-Cache. VLA-Pruner requires storing historical attention, but it introduces a negligible increase in Max GPU memory (see Appendix B.5.1).

5.3. More Results

Ablation Studies. To validate our design, we evaluate three alternative token selection criteria: (1) *prefill-only*, relying solely on prefill attention; (2) *action-only*, utilizing only action decode attention; and (3) *score-fusion*, a naive weighted linear combination of both. As shown in Figure 6,

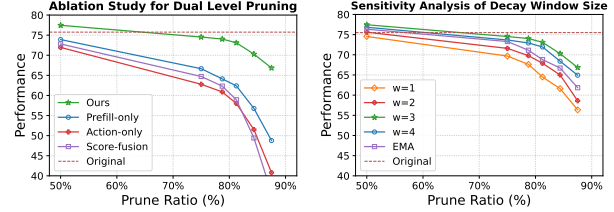


Figure 6. Ablation study to validate our dual-level design and Sensitive Analysis of decay window size. The results verify the sound design and robustness of VLA-Pruner.

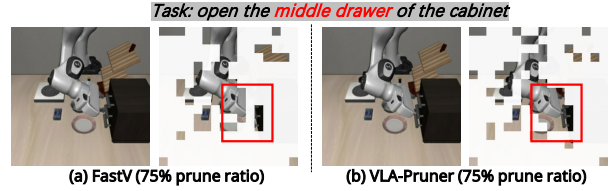


Figure 7. Qualitative case study of VLA-Pruner. VLA-Pruner preserves essential localized visual information for precise action execution, while maintaining broad semantic understanding.

prefill-only preserves high-level semantics but hinders fine-grained low-level control, whereas *action-only* improves short-horizon control but sacrifices semantic grounding and task planning, and thus neither maintains performance well.

Sensitivity Analysis We analyze the sensitivity of VLA-Pruner to the decay window size w in Figure 6, which demonstrates the robustness of our method. We further study varying decay rate γ and pruning layer K in Appendix B.5.2.

Qualitative Visualization. We provide a qualitative case study of VLA-Pruner in Figure 7. VLA-Pruner preserves essential localized visual information for precise action execution, while maintaining broad semantic understanding.

6. Conclusion

We propose VLA-Pruner, a training-free token pruning method for efficient VLA inference. We reveal the dual-system nature of VLA models, exposing fundamental limitations in current pruning approaches. Considering this, VLA-Pruner employs a dual-objective importance criterion that integrates semantic-level relevance and action-level importance. Based on this, our proposed dual-level token selection strategy adaptively retains visual tokens. Across diverse robotic manipulation tasks and multiple VLA architectures, VLA-Pruner reduces computation while preserving performance. We discuss our limitations in Appendix C.

References

- Agrawal, H., Desai, K., Wang, Y., Chen, X., Jain, R., Johnson, M., Batra, D., Parikh, D., Lee, S., and Anderson, P. Nocaps: Novel object captioning at scale. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 8948–8957, 2019.
- Alvar, S. R., Singh, G., Akbari, M., and Zhang, Y. Divprune: Diversity-based visual token pruning for large multimodal models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 9392–9401, 2025.
- Bai, J., Bai, S., Yang, S., Wang, S., Tan, S., Wang, P., Lin, J., Zhou, C., and Zhou, J. Qwen-vl: A frontier large vision-language model with versatile abilities. *arXiv preprint arXiv:2308.12966*, 1(2):3, 2023.
- Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Box, G. E., Jenkins, G. M., Reinsel, G. C., and Ljung, G. M. *Time series analysis: forecasting and control*. John Wiley & Sons, 2015.
- Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- Chen, L., Zhao, H., Liu, T., Bai, S., Lin, J., Zhou, C., and Chang, B. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*, pp. 19–35. Springer, 2024.
- Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., and Song, S. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11): 1684–1704, 2025.
- Contreras, J., Espinola, R., Nogales, F. J., and Conejo, A. J. Arima models to forecast next-day electricity prices. *IEEE transactions on Power Systems*, 18(3):1114–1121, 2003.
- Cui, C., Ding, P., Song, W., Bai, S., Tong, X., Ge, Z., Suo, R., Zhou, W., Liu, Y., Jia, B., et al. Openhelix: A short survey, empirical analysis, and open-source dual-system vla model for robotic manipulation. *arXiv preprint arXiv:2505.03912*, 2025.
- Firoozi, R., Tucker, J., Tian, S., Majumdar, A., Sun, J., Liu, W., Zhu, Y., Song, S., Kapoor, A., Hausman, K., et al. Foundation models in robotics: Applications, challenges, and the future. *The International Journal of Robotics Research*, 44(5):701–739, 2025.
- Gardner, E. S. Exponential smoothing: The state of the art. *Journal of Forecasting*, 4(1):1–28, 1985.
- Gardner Jr, E. S. Exponential smoothing: The state of the art—part ii. *International journal of forecasting*, 22(4): 637–666, 2006.
- Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., and Gao, J. Model tells you what to discard: Adaptive kv cache compression for llms. *arXiv preprint arXiv:2310.01801*, 2023.
- Han, B., Kim, J., and Jang, J. A dual process vla: Efficient robotic manipulation leveraging vlm. *arXiv preprint arXiv:2410.15549*, 2024.
- Huang, C.-P., Wu, Y.-H., Chen, M.-H., Wang, Y.-C. F., and Yang, F.-E. Thinkact: Vision-language-action reasoning via reinforced visual latent planning. *arXiv preprint arXiv:2507.16815*, 2025.
- Hudson, D. A. and Manning, C. D. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 6700–6709, 2019.
- Hyndman, R. J. and Athanasopoulos, G. *Forecasting: principles and practice*. OTexts, 2018.
- Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanke, P., et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Kim, M. J., Finn, C., and Liang, P. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025a.
- Kim, M. J., Finn, C., and Liang, P. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025b.
- Li, Q., Liang, Y., Wang, Z., Luo, L., Chen, X., Liao, M., Wei, F., Deng, Y., Xu, S., Zhang, Y., et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024a.
- Li, X., Liu, M., Zhang, H., Yu, C., Xu, J., Wu, H., Cheang, C., Jing, Y., Zhang, W., Liu, H., et al. Vision-language foundation models as effective robot imitators. *arXiv preprint arXiv:2311.01378*, 2023.

- Li, X., Hsu, K., Gu, J., Pertsch, K., Mees, O., Walke, H. R., Fu, C., Lunawat, I., Sieh, I., Kirmani, S., Levine, S., Wu, J., Finn, C., Su, H., Vuong, Q., and Xiao, T. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024b.
- Li, Y., Song, Y., Cao, L., Tetreault, J., Goldberg, L., Jaimes, A., and Luo, J. Tgif: A new dataset and benchmark on animated gif description. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4641–4650, 2016.
- Li, Y., Meng, Y., Sun, Z., Ji, K., Tang, C., Fan, J., Ma, X., Xia, S., Wang, Z., and Zhu, W. Sp-vla: A joint model scheduling and token pruning approach for vla model acceleration. *arXiv preprint arXiv:2506.12723*, 2025.
- Lin, T., Zhong, Y., Du, Y., Zhang, J., Liu, J., Chen, Y., Gu, E., Liu, Z., Cai, H., Zou, Y., Zou, L., Zhou, Z., Li, G., and Zhao, B. Evo-1: Lightweight vision-language-action model with preserved semantic alignment, 2025a. URL <https://arxiv.org/abs/2511.04555>.
- Lin, Z., Lin, M., Lin, L., and Ji, R. Boosting multimodal large language models with visual tokens withdrawal for rapid inference. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 5334–5342, 2025b.
- Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023a.
- Liu, H., Li, C., Wu, Q., and Lee, Y. J. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023b.
- Liu, Y., Duan, H., Zhang, Y., Li, B., Zhang, S., Zhao, W., Yuan, Y., Wang, J., He, C., Liu, Z., et al. Mmbench: Is your multi-modal model an all-around player? In *European conference on computer vision*, pp. 216–233. Springer, 2024.
- Ma, Y., Song, Z., Zhuang, Y., Hao, J., and King, I. A survey on vision-language-action models for embodied ai. *arXiv preprint arXiv:2405.14093*, 2024.
- Mao, J., Shen, Y., Guo, J., Yao, Y., and Hua, X. Efficient token compression for vision transformer with spatial information preserved. *arXiv preprint arXiv:2503.23455*, 2025a.
- Mao, J., Shen, Y., Guo, J., Yao, Y., Hua, X., and Shen, H. Prune and merge: Efficient token compression for vision transformer with spatial information preserved. *IEEE Transactions on Multimedia*, 2025b.
- Martí, R., Martínez-Gavara, A., Pérez-Peló, S., and Sánchez-Oro, J. A review on discrete diversity and dispersion maximization from an or perspective. *European Journal of Operational Research*, 299(3):795–813, 2022.
- Mhammed, N. B., Saeed, S. A. M., and Ahmed, A. M. Eog signal modeling using double exponential smoothing for robot arm control system. *Kurdistan Journal of Applied Research*, 1(3):6–10, 2016.
- O’Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., Mandekar, A., Jain, A., et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6892–6903. IEEE, 2024.
- Park, S., Kim, H., Jeon, W., Yang, J., Jeon, B., Oh, Y., and Choi, J. Quantization-aware imitation-learning for resource-efficient robotic control. *arXiv preprint arXiv:2412.01034*, 2024.
- Peng, H., Long, F., and Ding, C. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on pattern analysis and machine intelligence*, 27(8):1226–1238, 2005.
- Plummer, B. A., Wang, L., Cervantes, C. M., Caicedo, J. C., Hockenmaier, J., and Lazebnik, S. Flickr30k entities: Collecting region-to-phrase correspondences for richer image-to-sentence models. In *Proceedings of the IEEE international conference on computer vision*, pp. 2641–2649, 2015.
- Porumbel, D. C., Hao, J.-K., and Glover, F. A simple and effective algorithm for the maxmin diversity problem. *Annals of Operations Research*, 186(1):275–293, 2011.
- Resende, M. G., Martí, R., Gallego, M., and Duarte, A. Grasp and path relinking for the max–min diversity problem. *Computers & Operations Research*, 37(3):498–508, 2010.
- Shang, Y., Cai, M., Xu, B., Lee, Y. J., and Yan, Y. Llava-prumerge: Adaptive token reduction for efficient large multimodal models. *arXiv preprint arXiv:2403.15388*, 2024.
- Sloman, S. A. The empirical case for two systems of reasoning. *Psychological bulletin*, 119(1):3, 1996.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

- Wang, H., Xu, J., Pan, J., Zhou, Y., and Dai, G. Specprune-vla: Accelerating vision-language-action models via action-aware self-speculative pruning. *arXiv preprint arXiv:2509.05614*, 2025.
- Wen, J., Zhu, Y., Li, J., Zhu, M., Tang, Z., Wu, K., Xu, Z., Liu, N., Cheng, R., Shen, C., et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *IEEE Robotics and Automation Letters*, 2025.
- Winters, P. R. Forecasting seasonals and trends by exponentially weighted moving averages. *Management science*, 6 (3):324–342, 1960.
- Xing, L., Huang, Q., Dong, X., Lu, J., Zhang, P., Zang, Y., Cao, Y., He, C., Wang, J., Wu, F., et al. Pyramid-drop: Accelerating your large vision-language models via pyramid visual redundancy reduction. *arXiv preprint arXiv:2410.17247*, 2024.
- Xu, D., Zhao, Z., Xiao, J., Wu, F., Zhang, H., He, X., and Zhuang, Y. Video question answering via gradually refined attention over appearance and motion. In *Proceedings of the 25th ACM international conference on Multimedia*, pp. 1645–1653, 2017.
- Xu, S., Wang, Y., Xia, C., Zhu, D., Huang, T., and Xu, C. Vla-cache: Efficient vision-language-action manipulation via adaptive token caching. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Yang, Y., Wang, Y., Wen, Z., Zhongwei, L., Zou, C., Zhang, Z., Wen, C., and Zhang, L. EfficientVLA: Training-free acceleration and compression for vision-language-action models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=SELY1DHZk2>.
- Ye, W., Wu, Q., Lin, W., and Zhou, Y. Fit and prune: Fast and training-free visual token pruning for multi-modal large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 22128–22136, 2025.
- Yu, Z., Wang, B., Zeng, P., Zhang, H., Zhang, J., Gao, L., Song, J., Sebe, N., and Shen, H. T. A survey on efficient vision-language-action models, 2025. URL <https://arxiv.org/abs/2510.24795>.
- Yue, Y., Wang, Y., Kang, B., Han, Y., Wang, S., Song, S., Feng, J., and Huang, G. Deer-vla: Dynamic inference of multimodal large language models for efficient robot execution. *Advances in Neural Information Processing Systems*, 37:56619–56643, 2024.
- Zhang, Y., Fan, C.-K., Ma, J., Zheng, W., Huang, T., Cheng, K., Gudovskiy, D., Okuno, T., Nakata, Y., Keutzer, K., et al. Sparsevlm: Visual token sparsification for efficient vision-language model inference. *arXiv preprint arXiv:2410.04417*, 2024.
- Zhang, Z., Sheng, Y., Zhou, T., Chen, T., Zheng, L., Cai, R., Song, Z., Tian, Y., Ré, C., Barrett, C., et al. H2o: Heavy-hitter oracle for efficient generative inference of large language models. *Advances in Neural Information Processing Systems*, 36:34661–34710, 2023.
- Zhao, Q., Lu, Y., Kim, M. J., Fu, Z., Zhang, Z., Wu, Y., Li, Z., Ma, Q., Han, S., Finn, C., et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pp. 1702–1713, 2025.
- Zitkovich, B., Yu, T., Xu, S., Xu, P., Xiao, T., Xia, F., Wu, J., Wohllhart, P., Welker, S., Wahid, A., et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pp. 2165–2183. PMLR, 2023.
- Zou, X., Lu, D., Wang, Y., Yan, Y., Lyu, Y., Zheng, X., Zhang, L., and Hu, X. Don’t just chase “highlighted tokens” in mllms: Revisiting visual holistic context retention. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

A. Method Details

A.1. Algorithm Outlines

We detail VLA-Pruner in Algorithm 1 (overall pipeline) and Algorithm 2 (min-redundancy filtering).

A.2. Complexity Analysis

Following FastV (Chen et al., 2024), SparseVLM (Zhang et al., 2024) and DivPrune (Alvar et al., 2025), we estimate the computational requirement of VLA-Pruner in terms of Transformer FLOPs, and focus on how pruning visual tokens changes the effective sequence length seen by each layer. As in prior work (Alvar et al., 2025), we count the FLOPs of multi-head attention and feed-forward networks (FFNs) in the backbone (the overhead of token selection as negligible compared to the main model FLOPs.) Let T be the total number of Transformer layers in the VLA backbone and K the index of the pruning layer (we prune tokens before the K layer). Denote the numbers of text and visual tokens before pruning by N and M ($M \gg N$), and let $\mu = N + M$ be the full sequence length. After VLA-Pruner keeps a fraction ρ of the visual tokens, the sequence length becomes $\tilde{\mu} = N + \rho M$. Following (Chen et al., 2024), the FLOPs of attention and FFN for one layer with sequence length n , hidden size d , and FFN intermediate size m can be approximated as

$$C(n) = 4nd^2 + 2n^2d + 2ndm. \quad (10)$$

Without pruning, the total FLOPs are

$$\text{FLOPs}_{\text{full}} = TC(\mu). \quad (11)$$

With VLA-Pruner, the first $K - 1$ layers still use the full sequence μ , while the remaining $T - K + 1$ layers use the shortened sequence $\tilde{\mu}$, giving

$$\text{FLOPs}_{\text{prune}} = (K - 1)C(\mu) + (T - K + 1)C(\tilde{\mu}). \quad (12)$$

Thus, the theoretical FLOPs ratio is

$$r_{\text{FLOPs}} = \frac{\text{FLOPs}_{\text{prune}}}{\text{FLOPs}_{\text{full}}} = \frac{(K - 1)C(\mu) + (T - K + 1)C(\tilde{\mu})}{TC(\mu)}. \quad (13)$$

Since the attention term in $C(\cdot)$ is quadratic in sequence length, the reduction scales roughly with ρ^2 in layers after K . This matches the empirical FLOPs ratios in Tab. 1: VLA-Pruner reduces FLOPs to about 60%, 40%, and 30% of the original model at 50%, 25%, and 12.5% token retention, respectively. Temporal smoothing over a short window of action-to-vision attention has complexity $O(wM)$ per timestep, and the greedy Max-Min filtering over the dual-level candidate pool of size $O(\rho M)$ is $O((\rho M)^2)$ per prefill. In practice, these components contribute little to the total wall-clock time, as these operations are invoked only once during the prefill stage. Finally, Eq. (13) shows

that VLA-Pruner offers a flexible way to trade accuracy for efficiency: the retained-token ratio $\rho \in (0, 1]$. Smaller ρ reduces FLOPs quadratically in layers after K , yielding higher speedup, while larger ρ recovers the original model’s performance. In our experiments, we report results at $\rho \in 0.5, 0.25, 0.125$, but VLA-Pruner can be used at any intermediate ratio to meet a given compute or latency target.

B. Experiments Details

B.1. Setting up Details

B.1.1. VLA MODEL DETAILS

We consider three representative VLA architectures in our experiments: OpenVLA, OpenVLA-OFT, and π_0 . All models take RGB observations and a natural language instruction as input, and produce robot actions end-to-end.

OpenVLA. OpenVLA (Kim et al., 2024) is a 7B-parameter open-source vision–language–action model built by fine-tuning a Prismatic VLM (Llama 2 language backbone with DINOv2 and SigLIP visual encoders) on robot manipulation episodes from the Open X-Embodiment dataset. The model represents actions as discrete tokens and decodes them autoregressively, enabling a single policy to control multiple embodiments out-of-the-box. OpenVLA serves as our base autoregressive VLA, and we follow its official checkpoints and evaluation settings on LIBERO.

OpenVLA-OFT. OpenVLA-OFT (Kim et al., 2025b) is an “Optimized Fine-Tuning” variant of OpenVLA that improves both control performance and inference efficiency. It augments the original decoder with parallel decoding and action chunking, and switches to a continuous action representation trained with a simple L1 regression objective. This substantially boosts success rates on LIBERO while increasing action-generation throughput. To be noted, OpenVLA-OFT utilizes bidirectional attention, meaning that during inference, there is already an action-to-vision cross-attention. However, we observe that during the early-layer prefill phase, the attention still focuses on semantic understanding, making action-to-vision less significant. In deeper layers, the action-to-vision attention starts to exhibit more distinct properties. This allows our method to effectively enhance token pruning performance by utilizing past timestep action-to-vision cross-attention at the deeper layers to guide current token pruning.

π_0 Model. π_0 (Black et al., 2024) is a prototype generalist robot policy that augments a pretrained vision–language model (PaliGemma backbone) with a continuous-action flow-matching head. Built on top of a large VLM to inherit Internet-scale semantic knowledge, π_0 is trained on diverse real-robot data and is designed for high-frequency control in dexterous manipulation tasks. The model decodes continuous motor commands via a flow-matching process with 10 integration steps. We use π_0 as a diffusion-based VLA back-

Algorithm 1 Overall Pipeline of VLA-Pruner

```

1: Input: Visual embeddings  $\mathbf{E}_v$ , Semantic attention scores  $\mathcal{S}_{v1}$ , Action attention history  $\{\mathcal{S}_{\text{act}}^{t-i}\}_{i=1}^w$ , Token budget  $\tilde{M}$ , Decay params  $\gamma$ 
2: Output: Selected index set  $\tilde{\mathcal{C}} \subseteq \{1, \dots, M\}$ , with  $|\tilde{\mathcal{C}}| = \tilde{M}$ 
3: if  $\tilde{M} \geq M$  then
4:   return  $\{1, \dots, M\}$  ▷ Skipping pruning
5: end if
6:  $\hat{\mathcal{S}}_{\text{act}} \leftarrow \sum_{i=1}^w \gamma^i \mathcal{S}_{\text{act}}^{t-i} / \sum_{i=1}^w \gamma^i$  ▷ Action Attention Estimation via Temporal Smoothing (Eq. 7)
7:  $\mathcal{C}_{v1} \leftarrow$  indices of Top- $\tilde{M}$  elements of  $\mathcal{S}_{v1}$ 
8:  $\mathcal{C}_{\text{act}} \leftarrow$  indices of Top- $\tilde{M}$  elements of  $\hat{\mathcal{S}}_{\text{act}}$ 
9: Max-Relevance pooling:  $\mathcal{C}_{\text{dual}} \leftarrow \mathcal{C}_{v1} \cup \mathcal{C}_{\text{act}}$ 
10: Min-Redundancy filtering:  $\tilde{\mathcal{C}} \leftarrow \text{REDUNDANCY-MINIMIZATION FILTERING}(\mathbf{E}_v, \mathcal{C}_{\text{dual}}, \tilde{M})$  ▷ Alg. 2
11: return  $\tilde{\mathcal{C}}$ 

```

Algorithm 2 Redundancy-Minimization Filtering

```

1: Input: Visual embeddings  $\mathbf{E}_v = \{v_1, \dots, v_M\}$ , candidate index set  $\mathcal{C}_{\text{dual}} \subseteq \{1, \dots, M\}$ , target size  $\tilde{M}$ 
2: Output: Selected index set  $\tilde{\mathcal{C}}$ , with  $|\tilde{\mathcal{C}}| = \tilde{M}$ 
3: if  $|\mathcal{C}_{\text{dual}}| \leq \tilde{M}$  then
4:   return  $\mathcal{C}_{\text{dual}}$ 
5: end if
6: Extract candidate features:  $\{v_i\}_{i \in \mathcal{C}_{\text{dual}}}$ 
7: Normalize:  $u_i \leftarrow v_i / \|v_i\|_2$  for all  $i \in \mathcal{C}_{\text{dual}}$ 
8: Compute cosine distance matrix  $D$  over  $\mathcal{C}_{\text{dual}}$ :  $D_{ij} \leftarrow 1 - u_i^\top u_j$ 
9:  $\tilde{\mathcal{C}} \leftarrow \emptyset$  ▷ Initialize by maximal second-nearest distance
10: for each  $i \in \mathcal{C}_{\text{dual}}$  do
11:   Let  $\{d_{i1}, d_{i2}, \dots\}$  be  $D_{ij}$  for  $j \in \mathcal{C}_{\text{dual}}, j \neq i$  sorted increasingly
12:    $s_i \leftarrow d_{i2}$  ▷ Second smallest distance
13: end for
14:  $i^* \leftarrow \arg \max_{i \in \mathcal{C}_{\text{dual}}} s_i$ 
15:  $\tilde{\mathcal{C}} \leftarrow \tilde{\mathcal{C}} \cup \{i^*\}$ 
16: while  $|\tilde{\mathcal{C}}| < \tilde{M}$  do
17:   for each  $j \in \mathcal{C}_{\text{dual}} \setminus \tilde{\mathcal{C}}$  do
18:      $m_j \leftarrow \min_{i \in \tilde{\mathcal{C}}} D_{ij}$  ▷ Min distance to selected set
19:   end for
20:    $j^* \leftarrow \arg \max_{j \in \mathcal{C}_{\text{dual}} \setminus \tilde{\mathcal{C}}} m_j$ 
21:    $\tilde{\mathcal{C}} \leftarrow \tilde{\mathcal{C}} \cup \{j^*\}$ 
22: end while
23: return  $\tilde{\mathcal{C}}$ 

```

bone to evaluate the cross-architecture generalizability of VLA-Pruner. To capture informative interactions and reduce noise, we record the action-to-vision cross-attention scores averaged over the latter half of these integration steps.

B.1.2. ACCELERATION BASELINE DETAILS

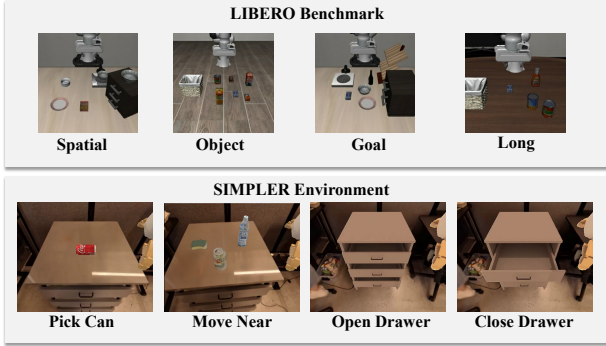
We briefly summarize the baseline acceleration methods considered in our experiments.

FastV. FastV (Chen et al., 2024)(ECCV24) accelerates large vision-language models by pruning redundant visual tokens based on early-layer prefill attention. Its core observation is that many image patches receive consistently low attention in deeper layers while contributing substantially to computation. FastV defines an attention-based importance score during the prefill stage and drops a fixed fraction of low-score visual tokens after a chosen layer. It is training-free and purely semantic, but does not account for the action-

dependent requirements of VLA models.

SparseVLM. SparseVLM (Zhang et al., 2024)(ICML25) also targets visual token redundancy in VLMs, but uses text-to-vision cross-attention as the importance signal. Tokens that receive low cross-attention from text queries are considered less relevant and pruned. Compared to FastV, SparseVLM focuses directly on aligning visual tokens with language semantics, yet still operates only at the semantic level and ignores the distinct visual needs of action decoding in VLA.

DivPrune. DivPrune (Alvar et al., 2025)(CVPR25) addresses the tendency of attention-based methods to select similar and redundant tokens. Instead of ranking tokens by attention scores, it formulates token selection as a Max-Min Diversity Problem and chooses a subset of visual tokens that are maximally dissimilar in feature space, thereby reducing redundancy while maintaining coverage. However,



(a) Tasks on LIBERO benchmark and the SIMPLER environment.

DivPrune is agnostic to the dual-level relevance of visual tokens in VLA prefilling and action decoding, and its diversity objective falls short for VLA tasks where localized details are critical.

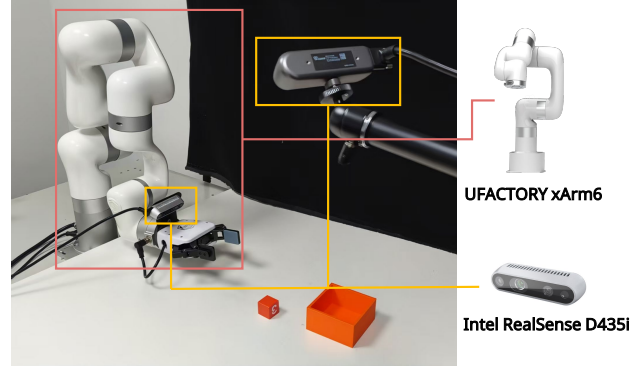
VLA-Cache. VLA-Cache (Xu et al., 2025)(Neurips25) is a training-free acceleration method tailored for VLA models. It exploits temporal continuity in robotic manipulation by identifying visually static tokens whose features change little across timesteps, and reuses their computation via a KV-cache. At the same time, it uses text-to-vision cross-attention to keep task-relevant tokens fully computed. This cache-based mechanism reduces repeated computation over static background regions, but its semantic attention score based token selection remains coarse-grained, which overlooks distinct action attention distribution. To ensure a fair comparison, we set the dynamic token limit to be $1.5 \times k$ (k is the token budget), and use text-to-vision score to exclude $0.5 \times k$ of the most relevant tokens. These settings follow the default setup of VLA-Cache codebase.

EfficientVLA. EfficientVLA (Yang et al., 2025)(Neurips25) proposes a structured acceleration framework for diffusion-based VLA models that combines visual token pruning, layer reduction, and diffusion temporal feature caching. Its token-pruning component leverages semantic attention and diversity heuristics inherited from VLM pruning, while additional modules (i.e., layer skipping and diffusion caching) are used to achieve comparable acceleration. In contrast, VLA-Pruner is a single, plug-and-play token pruning module. These approaches are therefore complementary: EfficientVLA and other VLA acceleration methods can be adopted orthogonally with VLA-Pruner, while VLA-Pruner achieves strongest token-prune performance.

B.2. Benchmark Details

Fig. 8a shows the evaluation simulation environments and Fig. 8b presents our real-world xArm6 robot setup.

LIBERO. LIBERO (Liu et al., 2023a) is a Robosuite-based benchmark for lifelong robot manipulation that groups



(b) Real-world xArm6 robot setup.

130 language-conditioned tasks into four suites: *Spatial*, *Object*, *Goal*, and *Long*. LIBERO-Spatial varies only the spatial configuration of otherwise identical objects (e.g., pick up the black bowl on the stove and place it on the plate). LIBERO-Object varies the manipulated object while keeping the scene layout and goal template fixed (e.g., pick up the bbq sauce and place it in the basket). LIBERO-Goal keeps both the objects and spatial layout fixed but changes the goal description (e.g., open the middle drawer of the cabinet). LIBERO-Long corresponds to the long-horizon subset (10 tasks) of LIBERO-100, where each episode consists of multi-stage instructions such as opening a drawer, inserting an object, and then rearranging other items (e.g., put both the alphabet soup and the cream cheese box in the basket).

SIMPLER. SIMPLER (Li et al., 2024b) is a simulation suite built to closely track the real Google robot environment while allowing controlled appearance variation. It provides two evaluation settings. The *Visual Matching* (VM) setting keeps backgrounds, lighting, distractors, and camera poses close to the real setup so that rendered observations visually resemble real robot scenes and better predict real-world performance. The *Variant Aggregation* (VA) setting starts from the same base scenes but procedurally perturbs nuisance factors such as background textures, illumination, number and placement of distractor objects, table material, and camera viewpoints, creating a broad distribution shift for testing robustness. For the Google robot configuration, SIMPLER defines four canonical manipulation tasks: “pick coke can”, “pick object”, “move near”, “open/close Drawer”, and “place in closed drawer”. In our experiments, we follow the official simulator settings and success metrics, and select three representative tasks—*Pick Coke Can*, *Move Near*, and *Open/Close Drawer*—under VM or VA setting, where base model attains non-trivial performance.

Real robot. As shown in Figure 8b, our real-world system is built on a 6-DoF xArm6 manipulator equipped with two Intel RealSense D435i RGB cameras: one mounted on the wrist for close-range observations and the other fixed for capturing global scene context. The robot is controlled

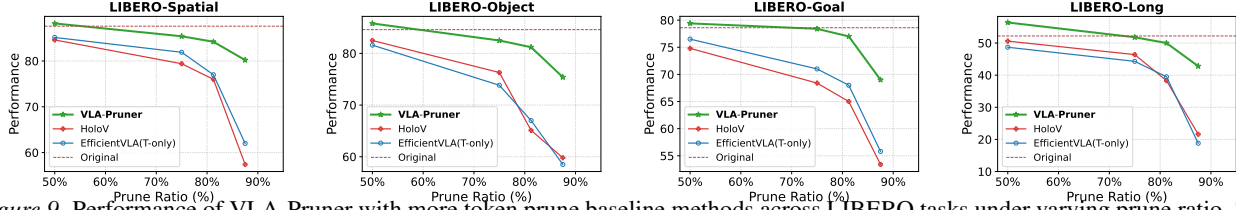


Figure 9. Performance of VLA-Pruner with more token prune baseline methods across LIBERO tasks under varying prune ratio. The horizontal axis represents the pruning/caching ratios of visual tokens, and the vertical axis shows the success rates.

Table 3. Performance comparison with various acceleration methods.

Method	Spatial	Object	Goal	Long	Avg.↑	FLOPs↓	Speedup↑
VLA-Pruner	85.4	82.5	78.4	51.8	74.5	39.80%	1.633×
EfficientVLA (T-only)	84.3	81.6	76.5	49.8	73.1	58.12%	1.334×
EfficientVLA	83.2	80.3	75.7	49.1	72.0	41.30%	1.491×
VTW	74.7	76.2	72.3	48.9	68.0	55.17%	1.325×
FitPrune	82.4	80.1	74.8	50.3	71.9	47.71%	1.397×

through the official xArmAPI. We collect real-world demonstrations following the LeRobot2.1 data specification. Each episode consists of synchronized observations recorded at 30Hz, including two RGB image streams. The proprioceptive state is represented by the robot’s absolute joint angles, and actions are stored using the same absolute joint angles to ensure consistent replay and supervision. All demonstrations are gathered using this unified multi-view setup, which provides both global scene context and fine-grained manipulation details that are essential for learning robust visuomotor policies.

B.3. Results of More Baselines

Varying pruning ratio. Besides the main training-free baselines in the paper (FastV (Chen et al., 2024), SparseVLM (Zhang et al., 2024), DivPrune (Alvar et al., 2025), VLA-Cache (Xu et al., 2025)), we evaluate two recent visual token pruning methods with controllable pruning ratios: token-pruning module of EfficientVLA (Yang et al., 2025), denoted as *EfficientVLA(T-only)* and HoloV(Neurips25) (Zou et al.). We evaluate them on OpenVLA backbone across four LIBERO suites. For each method, we report the average success rates under varying retained token ratios in Fig. 9. VLA-Pruner achieves consistent superiority across all suites. These results further support that semantic-only token pruning is ill-suited for VLA, and that our dual-level strategy better matches the requirements of VLA robotic control.

Comparison under comparable performance. We compare VLA-Pruner against structured and calibration-based baselines, focusing on the trade-off between VLA performance and computational cost. We consider the full EfficientVLA framework (Yang et al., 2025), which combines token pruning and layer reduction, as well as the calibration-based methods FitPrune (Ye et al., 2025) and VTW (Lin

et al., 2025b). For EfficientVLA, we set the visual-token retention at 50% and reduce 8 layers to ensure comparable performance while maximizing acceleration. We report VLA-Pruner at 25% token retention, which achieves the better performance while requiring fewer FLOPs and lower latency. This suggests that dual-level VLA token pruning can achieve a better efficiency–performance trade-off than EfficientVLA, which is designed primarily for diffusion head policy and overlook the intrinsic characteristics of VLA inference for token pruning, thereby showing limited performance. For FitPrune and VTW, we follow their official calibration protocols and perform calibration on 100 LIBERO tasks, which corresponds to a substantially larger calibration fraction than that typically used in VLM benchmarks. Both methods achieve lower success rates than VLA-Pruner at 25% token retention and consume more FLOPs (Tab. 3). VTW typically chooses a late withdrawal layer and degrades VLA performance, while FitPrune, though effective on standard VLM benchmarks, shows limited favorable trade-off in the VLA setting since it stills rely on semantic attention scores.

B.4. Results of π_0 on LIBERO

Setup. We evaluate VLA-Pruner on the π_0 model across LIBERO benchmark at various token retention ratios (50%, 25%, and 12.5%). For each configuration, we measure success rates, relative accuracy, latency, speedup, and FLOP ratios. We set the number of flow matching integration steps as 10, action chunk as 50 and execute 5 actions per step.

Main Results. VLA-Pruner consistently achieves high performance across all token retention settings. Notably, at each tested token retention ratio, our method maintains superior performance relative to other baselines, especially as pruning ratio increases. In particular, for π_0 , our method demonstrates impressive efficiency, maintaining comparable

Table 4. Performance of VLA-Pruner on π_0 across LIBERO at different vision-token retention ratios. We report success rates (%) on the four suites, relative accuracy, latency (ms), speedup ($\times$) and FLOP ratio (%).

Method	LIBERO Success (%)					Acc.(%)↑	Latency(ms)↓	Speedup (×)↑	FLOP ratio(%)↓
	Spatial	Object	Goal	Long	Avg.				
Upper Bound (100%)									
Vanilla	96.9	98.3	96.1	84.8	94.03	100.0	104.53	1.000	100
Retain 50% Tokens									
FastV	95.26	95.46	92.58	81.34	91.16	91.16	66.44	1.573	60.7
SparseVLM	94.70	93.96	91.50	79.48	89.91	89.91	70.39	1.485	61.5
DivPrune	91.30	92.14	88.25	77.19	87.22	87.22	69.17	1.511	58.9
VLA-Cache	96.47	95.99	96.59	84.65	93.43	93.43	78.30	1.335	72.3
VLA-Pruner	97.01	98.61	97.57	87.12	95.07	101.10	69.45	1.505	60.9
Retain 25% Tokens									
FastV	88.28	78.81	86.38	73.61	81.77	86.97	57.68	1.812	39.9
SparseVLM	89.63	84.78	81.80	72.67	82.22	86.76	61.09	1.711	41.3
DivPrune	84.41	71.60	80.61	66.60	75.80	80.76	59.05	1.770	39.3
VLA-Cache	85.60	85.32	85.94	73.97	82.71	87.86	64.81	1.613	51.4
VLA-Pruner	95.28	96.25	95.71	83.01	92.56	98.44	60.38	1.731	40.1
Retain 12.5% Tokens									
FastV	68.31	59.15	57.97	28.08	53.38	53.37	51.72	2.021	30.1
SparseVLM	72.04	64.98	63.55	30.99	57.89	57.89	53.79	1.943	31.3
DivPrune	63.41	63.78	62.79	29.61	54.90	54.90	52.64	1.986	29.8
VLA-Cache	59.85	58.69	64.71	27.09	52.58	55.92	59.02	1.771	39.4
VLA-Pruner	89.64	90.55	86.60	65.56	83.08	88.35	53.49	1.954	30.4

Table 5. Memory and Runtime Analysis of Acceleration Methods on OpenVLA. Detailed comparison of maximum GPU memory consumption and CUDA runtime across different vision-token retention ratios.

Method	50% Retained		25% Retained		12.5% Retained	
	Max Memory (GB) $\downarrow$	CUDA Time (ms) $\downarrow$	Max Memory (GB) $\downarrow$	CUDA Time (ms) $\downarrow$	Max Memory (GB) $\downarrow$	CUDA Time (ms) $\downarrow$
Vanilla	Origin Memory: 14.75 GB / Origin CUDA Latency: 51.43 ms					
FastV	14.513	29.022	14.327	25.58	14.311	24.33
SparseVLM	14.544	29.594	14.351	25.71	14.329	23.64
DivPrune	14.438	28.052	14.303	24.98	14.281	22.91
VLA-Cache	14.592	36.33	14.523	35.13	14.484	34.57
VLA-Pruner	14.527	29.192	14.348	25.58	14.322	23.35

success rates even at 12.5% token retention. This enables a significant acceleration in computation while preserving performance, a key advantage over traditional pruning methods. While VLA-Pruner’s redundancy filtering slightly affects acceleration performance on π_0 with high-frequency, the impact remains minimal. Even at a 25% retention ratio, VLA-Pruner surpasses other token pruning methods at 50% retention ratio, demonstrating that our method can precisely preserve essential information. The results in Table 4 clearly illustrate how VLA-Pruner maintains robust task success across pruning ratios, highlighting its cross-architecture generalizability to provide an superior trade-off between computational efficiency and model accuracy for diffusion-based model.

B.5. More results

B.5.1. DETAILED EFFICIENCY RESULTS

We report additional efficiency results, including peak GPU memory consumption (GB) and CUDA runtime (ms), across different vision-token retention ratios. As shown in Table 5, VLA-Pruner exhibits comparable efficiency to other pruning methods.

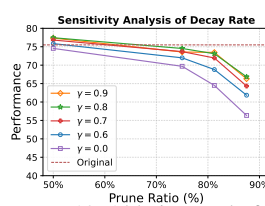


Figure 10. Ablation study for decay rate.

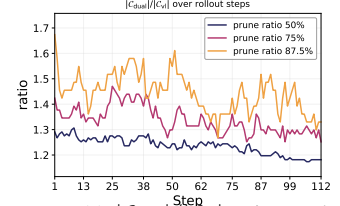


Figure 11. $|C_{\text{dual}}|/|C_{\text{vl}}|$ ratio trends at different pruning ratios on robot task.

Method	TFLOP (ratio %)	Spatial	Object	Goal	Long	Avg
K=2	37.90	69.7	74.1	55.3	34.8	58.48
K=3 (Ours)	39.77	85.4	82.5	78.4	51.8	74.53
K=4	41.98	85.5	82.3	78.1	52.0	74.48
K=5	43.08	83.9	81.7	77.9	51.4	73.73

Table 6. Study on applying pruning at different layers.

B.5.2. MORE ANALYSIS

Decay rate γ . We analyze how different decay rates $\gamma \in \{0.0, 0.6, 0.7, 0.8, 0.9\}$ influence VLA-Pruner and report the results for average success rates under varying prune ratio in Fig. 10.

Pruning layer K . We compare results of different pruning layer values $K \in \{2, 3, 4, 5\}$ in Table 6 (retain 25% tokens).

VLA-Pruner details. We show $|C_{\text{dual}}|/|C_{\text{vl}}|$ ratio trends at different pruning ratios for a robotic task in Fig. 11.

B.6. More Visualization

We provide detailed visualization of VLA-Pruner token prune results across LIBERO tasks (50%, 75% pruning ratios) in Fig. 12. VLA-Pruner precisely maintains both local details for action execution and global information for semantic understanding and task planning.

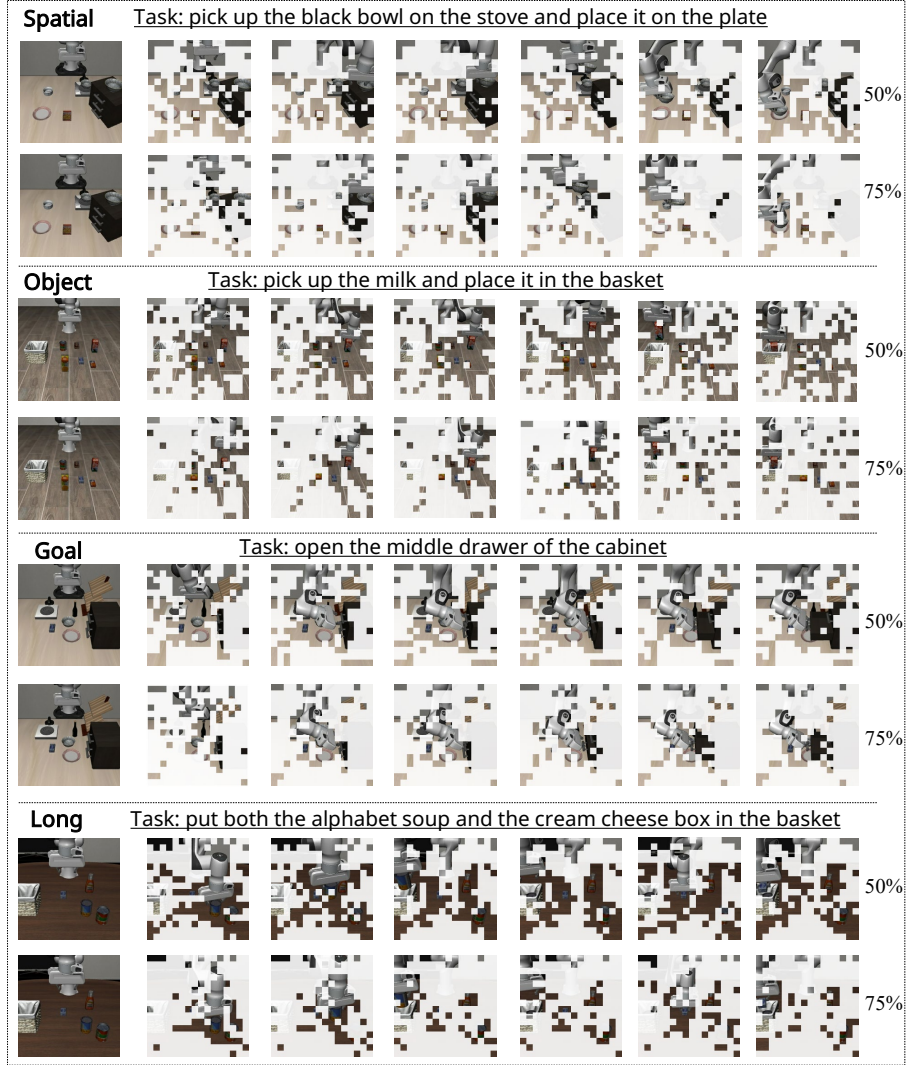


Figure 12. Visualization of VLA-Pruner token pruning results across LIBERO tasks under different ratios

C. Limitations and Future work.

Our main contribution is to point out the dual-system nature of VLA inference and to propose a dual-level importance criterion that jointly accounts for semantic and action-level relevance. However, our current estimation of action-level importance relies on a heuristic temporal smoothing scheme with a fixed window and decay rate. While this works well for most tabletop settings, its benefits may diminish in highly dynamic scenarios (e.g., egocentric wrist-camera views or conveyor-belt environments), where rapid viewpoint shifts and object motion can violate the short-term temporal continuity assumption and blur sharp changes in action-to-vision attention. A promising direction for future work is to replace the fixed temporal smoothing with an adaptive prediction module that adjusts its effective window based on motion cues or feature changes (e.g., magnitude of visual dynamics or action-conditioned attention variance), or to learn a lightweight temporal attention network that directly predicts per-token importance from recent history. Such adaptive mechanisms may further improve pruning quality in dynamic environments while retaining the advantages of VLA-Pruner.